

Lab0 实验报告

一、思考题

1. Git 文件状态管理

初始状态（无提交，未跟踪文件）

位于分支 `master`
尚无提交

未跟踪的文件：
（使用 `"git add <文件>..."` 以包含要提交的内容）
 `README.txt`
 `Untracked.txt`

提交为空，但是存在尚未跟踪的文件（使用 `"git add"` 建立跟踪）

执行 `git add .` 后

...
要提交的变更：
（使用 `"git rm --cached <文件>..."` 以取消暂存）
 新文件: `README.txt`
 新文件: `Untracked.txt`

未跟踪的文件：
（使用 `"git add <文件>..."` 以包含要提交的内容）
 `Stage.txt`

修改文件后

...
尚未暂存以备提交的变更：
（使用 `"git add <文件>..."` 更新要提交的内容）
（使用 `"git restore <文件>..."` 丢弃工作区的改动）
 修改: `README.txt`

未跟踪的文件：
（使用 `"git add <文件>..."` 以包含要提交的内容）
 `Modified.txt`
 `Stage.txt`

修改尚未加入提交（使用 `"git add"` 和/或 `"git commit -a"`）

结论

1. `Modified.txt` 初始状态为 **Untracked (未跟踪)**
2. 执行 `git add` 后, 文件进入 **Staged (已暂存)** 状态
3. 暂存/提交后再次修改文件, 状态变为 **modified (已修改)**
4. 文件被 `git add` 跟踪后, 无法自动回到 Untracked 状态, 需手动执行移除命令

2. Git 基础操作命令

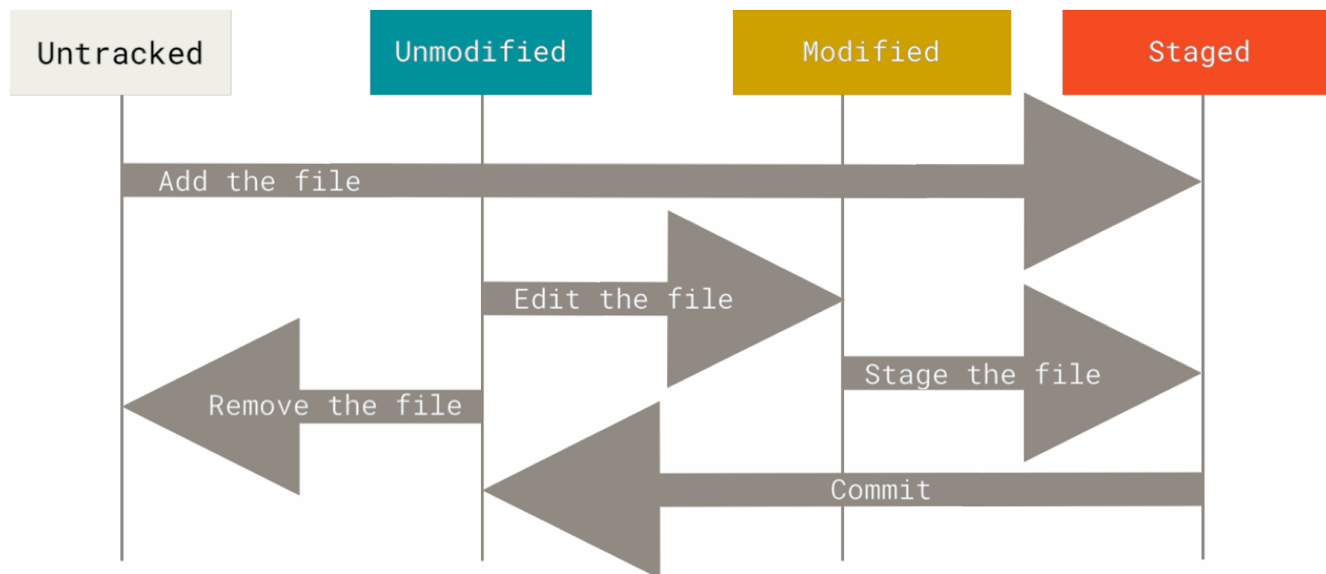


图 0.10: Git 中的四种状态转换关系

1. 添加文件到暂存区: `git add`
2. 提交暂存区内容: `git commit -m "备注信息"`

3. Git 文件恢复与暂存区操作

(1) 误删工作区文件 `print.c`, 恢复命令

```
git restore print.c
```

(2) 误删后执行 `git rm print.c`, 恢复命令

文件删除操作已同步至暂存区, 需从版本库恢复:

```
git reset HEAD print.c
# 新版 Git 推荐
git restore --staged print.c
```

(3) 取消暂存无关文件 `hello.txt` (不删除本地文件)

```
git rm --cached hello.txt
```

4. Git 版本回退

1. `git reset`: 回退到指定版本, `HEAD^` 表示回退到上一版本, 使用版本哈希值可任意切换版本
2. `git reset --hard`: 强制回退, **工作区内容会被直接覆盖, 所有修改会丢失**

5. Shell 输出重定向

```
git@24371277:~/learnGit (master)$ echo first
first
git@24371277:~/learnGit (master)$ echo second > output.txt
git@24371277:~/learnGit (master)$ cat output.txt
second
git@24371277:~/learnGit (master)$ echo third > output.txt
git@24371277:~/learnGit (master)$ cat output.txt
third
git@24371277:~/learnGit (master)$ echo fourth >> output.txt
git@24371277:~/learnGit (master)$ cat output.txt
third
fourth
```

- `>`: 覆盖式输出重定向
- `>>`: 追加式输出重定向 (文件末尾添加, 不覆盖原内容)

6. Shell 脚本综合实验

脚本文件 `command.sh`

```
#!/bin/bash
# 1. 使用 Here Document 创建 test 脚本
cat > test << 'EOF'
echo Shell Start...
echo set a = 1
a=1
echo set b = 2
b=2
echo set c = a+b
c=$((a+b))
echo c = $c
```

```
echo save c to ./file1
echo $c>file1
echo save b to ./file2
echo $b>file2
echo save a to ./file3
echo $a>file3
echo save file1 file2 file3 to file4
cat file1>file4
cat file2>>file4
cat file3>>file4
echo save file4 to ./result
cat file4>>result
EOF

# 2. 添加可执行权限
chmod +x test

# 3. 运行脚本，输出追加到 result
./test >> result
```

输出文件 `result.txt`

```
Shell Start...
set a = 1
set b = 2
set c = a+b
c = 3
save c to ./file1
save b to ./file2
save a to ./file3
save file1 file2 file3 to file4
save file4 to ./result
3
2
1
```

输出解释

1. 执行过程输出：展示 Shell 变量定义、算术运算、文件操作流程
2. 文件操作：通过重定向实现文件写入、内容拼接
3. 最终结果：`result` 文件包含脚本执行日志与运算结果

命令替换对比

```
echo echo Shell Start
echo `echo Shell Start`
```

1. 前者：直接输出字符串 `echo Shell Start`
2. 后者：命令替换，执行内部指令后输出结果 `Shell Start`

```
echo echo $c > file1
echo `echo $c` > file1`
```

1. 前者：向 `file1` 写入 `echo 10`
2. 后者：向 `file1` 写入 `10`，外层输出空行

二、难点分析与实验体会

1. Linux / Shell 命令学习

核心难点：将 GUI 操作转换为 CLI 命令，实现批处理自动化

- 熟练查阅指导文档，快速学习命令用法
- 灵活使用**管道与重定向**实现复杂逻辑

示例命令：将所有的**地址行**的地址**提取**，根据其**重复次数**，由大到小**排序**，将地址与重复次数**输出**到 `result.txt`中

```
awk '{print $1}' "$access_src" | uniq -c | sort -r -n > $summary_dst
```

2. Makefile 核心逻辑

Makefile 用于自动化构建项目，核心是**依赖关系管理**：

- 定义目标、依赖文件与编译指令
- 自动完成编译、运行、清理等操作
- 底层均为 Linux 系统命令

实验 Makefile 代码

```
include makefile.inc

.PHONY: run clean
COMMON_FILE = main.c post_calc.c common.h

run: casegen ver1 ver2
    ./casegen $(CASE_NUM) $(INPUT_FILE)
    ./ver1 $(INPUT_FILE)
    ./ver2 $(INPUT_FILE)

all: casegen ver1 ver2

ver1: $(COMMON_FILE) calc1.c
```

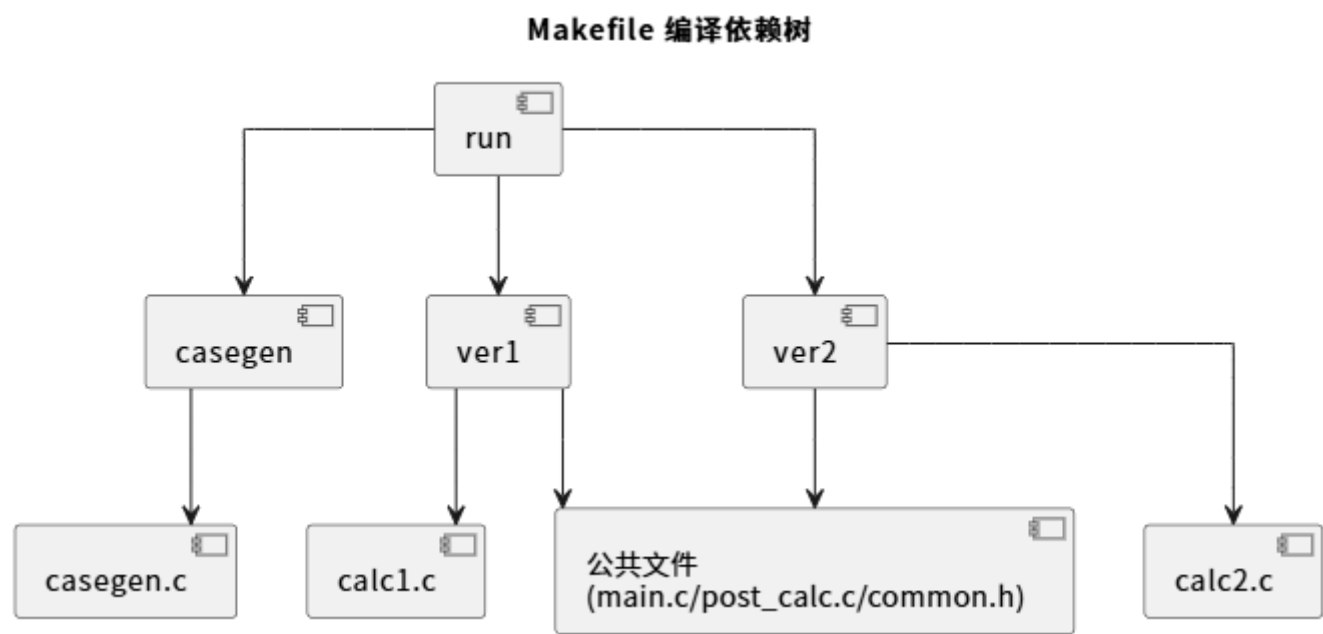
```
gcc $(COMMON_FILE) calc1.c -o ver1

ver2: $(COMMON_FILE) calc2.c
gcc $(COMMON_FILE) calc2.c -o ver2

casegen: casegen.c
gcc casegen.c -o casegen

clean:
# TODO: fill the clean command
rm -f casegen ver1 ver2 *.txt *.o
# TODO: setup PHONY rules
```

Makefile 编译依赖树



3. 实验注意事项

- 1. 避免在 Vim 编辑文件时执行 Git 提交，防止临时隐藏文件干扰
- 2. 学习 CLI / Vim 快捷键，提升操作效率

三、原创说明

参考资料：2026OS-guide-book.pdf